

Software: Python and Data Analytics

How a language and its ecosystem power data mining

Adem Yahmadi

Department of Informatics
Technische Universität München

Seminar Data Mining

Outline

- 1 Motivation
- 2 The Python Language
- 3 The Python Ecosystem
- 4 Data Mining Workflows
- 5 Live Demo
- 6 Conclusion

Why does the *software* matter?

- ▶ Modern systems and research generate enormous volumes of data.
- ▶ **Data mining** = uncovering hidden patterns that drive better predictions and decisions.
- ▶ It is not just *algorithms* — it is the *software* that processes, models and visualizes the data.

The language choice shapes the workflow

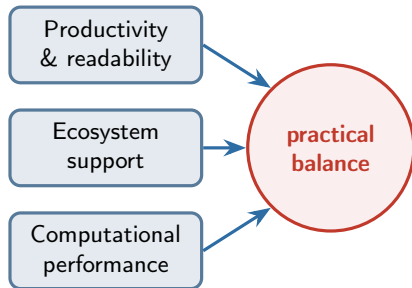
Compiled languages run faster, but higher-level languages buy **faster development** and **easier experimentation** — which, in data mining, usually matters more.

Why Python?

- ▶ Open-source, readable, fast to prototype.
- ▶ One of the dominant languages for data mining, ML and data science.
- ▶ A **broad ecosystem** spanning raw analysis all the way to deployment.

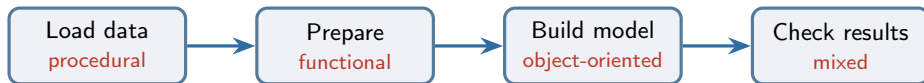
The trade-off

Interpreted & dynamically typed $\Rightarrow$ slower than compiled languages — but optimized libraries hide most of that cost.



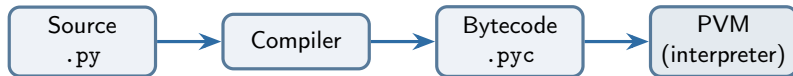
Paradigms & syntax: one workflow, several styles

Python is multi-paradigm — and data mining naturally mixes the styles:



- ▶ **Readability:** indentation instead of braces → easy to inspect and revise (crucial when code changes constantly).
- ▶ **Dynamic typing:** fast to write and experiment with, at the cost of runtime type checks.

Performance: how CPython runs your code



- ▶ CPython **compiles** .py source to platform-independent **bytecode** (cached as .pyc in `__pycache__`).
- ▶ The **PVM** then **interprets** that bytecode at runtime — the core overhead vs. compiled C/C++.

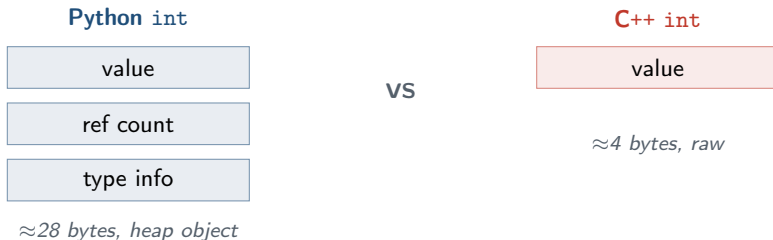
Global Interpreter Lock (GIL)

- ▶ A **mutex**: one thread runs Python bytecode at a time.
- ▶ **Why**: reference counting is not thread-safe without it.
- ▶ **Effect**: CPU-bound threads serialise — no multi-core gain.

Why it is usually fine

- ▶ **multiprocessing**: separate processes, each with its own GIL.
- ▶ **NumPy** releases the GIL during heavy numerical loops.
- ▶ **PEP 703/779**: optional no-GIL build (experimental 3.13, supported 3.14).

Performance: the memory overhead — and the fix

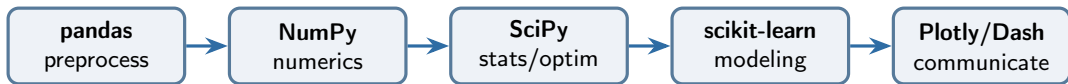


- ▶ Python integers are full *objects*: value + ref count + type info → more memory, worse cache locality.
- ▶ Reference counting + cyclic GC add runtime overhead on every assignment.

The escape hatch

NumPy packs raw values in a contiguous buffer and runs *vectorized* operations in compiled C — Python becomes a thin, high-level interface to fast native routines.

The ecosystem: one library per stage



- ▶ They are **not** isolated tools — they share data structures and interoperate.
- ▶ The same dataset can flow through all of them in *one* environment.

Running example throughout: `df = px.data.gapminder()` — life expectancy vs GDP, 1952–2007.

NumPy — array-based, vectorized computing

- ▶ Core type: the `ndarray` (homogeneous, contiguous memory).
- ▶ Speed from **vectorization**, **broadcasting** and **reductions** — all in compiled code, no Python loops.

```
import numpy as np
gdp = df["gdpPercap"].to_numpy()      # ndarray
log_gdp = np.log10(gdp)               # vectorize
span    = gdp.max() - gdp.min()       # reduce
scaled  = (gdp - gdp.min())/span      # broadcast
```

Limits: homogeneous & fixed-size; in-memory only (large/GPU → Dask, CuPy, PyTorch).

pandas — tabular data & preprocessing

- ▶ The DataFrame: labeled rows/columns, *mixed* types.
- ▶ Built for the messy reality: missing values, scaling, grouping.
- ▶ Label-based alignment keeps transforms consistent.

```
import plotly.express as px
df = px.data.gapminder()           # 1704 rows
df07 = df[df["year"] == 2007]      # filter
df07 = df07.dropna()               # clean
avg = df07.groupby("continent")["lifeExp"].mean()
```

Cost: more memory than NumPy (labels, object dtype); very large data → Dask / Spark.

SciPy — scientific & statistical computing

- ▶ Works *directly* on NumPy arrays.
- ▶ stats: hypothesis tests (e.g. two-sample t-test).
- ▶ optimize: minimize a loss — the heart of model fitting.

Least squares:

$$\min_{\theta_0, \theta_1} \sum_i (y_i - (\theta_0 + \theta_1 x_i))^2$$

```
from scipy import stats, optimize
import numpy as np
eu = df07.loc[df07.continent=="Europe", "lifeExp"]
af = df07.loc[df07.continent=="Africa", "lifeExp"]
t, p = stats.ttest_ind(eu, af)    # t-test

x = np.log10(df07["gdpPercap"])    # feature
y = df07["lifeExp"]               # target
def loss(theta):                  # squared err
    t0, t1 = theta
    return ((y - (t0 + t1*x))**2).sum()
res = optimize.minimize(loss, x0=[0, 1])
```

scikit-learn — the unified estimator API

- ▶ One consistent interface for everything:
- ▶ `fit` learns, transform maps, `predict` outputs.
- ▶ Makes preprocessing + models composable (Pipeline).

Classical ML only — no deep learning / autodiff (that is PyTorch, TensorFlow).

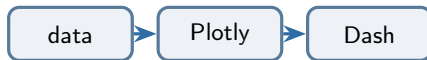
```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.log10(df07[["gdpPercap"]]) # feature
y = df07["lifeExp"]               # target
scaler = StandardScaler()
X_s = scaler.fit_transform(X)     # scale
model = LinearRegression()
model.fit(X_s, y)                 # train
y_pred = model.predict(X_s)      # predict
```

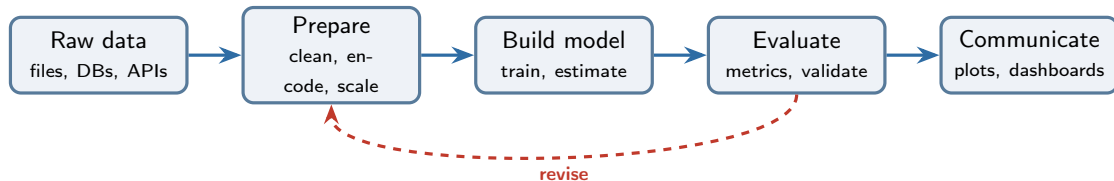
Plotly & Dash — communicate and deploy

- ▶ **Plotly**: interactive plots (zoom, hover) straight from a DataFrame.
- ▶ **Dash**: turn plots + controls into a web dashboard — no code for the end user.
- ▶ **FastAPI**: serve a trained model behind an API.

```
import plotly.express as px
fig = px.scatter(
    df07, x="gdpPerCap", y="lifeExp",
    size="pop", color="continent",
    log_x=True, hover_name="country")
fig.show()
```



Data mining is iterative, not a straight line

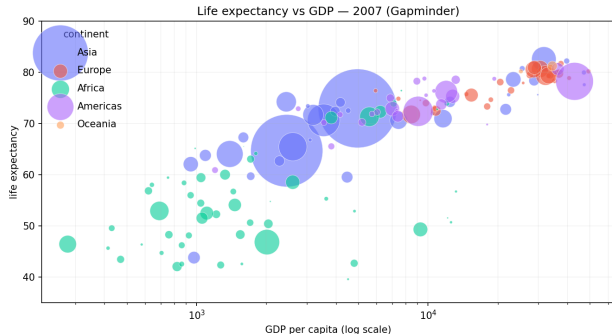


- ▶ Evaluation often sends you *back*: poor missing-value handling, weak features, or the wrong model.
- ▶ Python's strength = **integration**: every stage in one environment.
- ▶ **Reproducibility**: scripts, notebooks and pipelines let anyone re-run the whole workflow and get the same result.
- ▶ **In production**: keep watching for *drift* — when new, incoming data no longer looks like the data the model was trained on.

Live demo: an interactive dashboard (Dash + Plotly)

Gapminder — the same pipeline, made interactive:

- 1 **pandas** — load & filter the table.
- 2 **NumPy** — vectorized $\log_{10}(\text{GDP})$.
- 3 **SciPy** — t-test: rich vs poor half.
- 4 **scikit-learn** — fit $\text{lifeExp} \sim \log \text{GDP}$ (R^2).
- 5 **Plotly + Dash** — live bubble chart & year slider.



Takeaways

- ▶ Python's role rests on **both** the language *and* its ecosystem.
- ▶ Raw speed is limited — but optimized, compiled-backed libraries recover most of it.
- ▶ The libraries shine *together*, in an iterative workflow:

pandas → NumPy → SciPy → scikit-learn → Plotly/Dash

What to remember

Python's main strength in data mining is **integration** — it connects preprocessing, computation, analysis, modeling, evaluation and deployment into a single workflow.

Thank you!

Questions & discussion

Adem Yahmadi | adem.yahmadi@tum.de | TUM Informatics